

在 Android 裝置使用 TensorFlow Lite 辨識花朵

來源：<https://codelabs.developers.google.com/codelabs/recognize-flowers-with-tensorflow-on-android?hl=zh-tw>

步驟數：9

在本程式碼研究室中，您將擷取圖片分類器，並透過 TensorFlow Lite 在 Android 手機上執行。

目錄

1. [1. 簡介](#)
2. [2. 使用 Colab 訓練花卉辨識器](#)
3. [3. 設定工作目錄](#)
4. [4. 設定 Android 骨架應用程式](#)
5. [5. 將 TensorFlow Lite 新增至 Android 應用程式](#)
6. [6. 選用：查看所有待辦事項清單](#)
7. [7. 使用 TensorFlow Lite 執行自訂模型](#)
8. [8. 選用：使用 GPU 委派加速推論](#)
9. [9. 後續步驟](#)

1. 簡介

注意：本程式碼研究室使用 TensorFlow Lite 程式庫。建議所有新程式碼都使用 [Google AI Edge LiteRT 程式庫工具](#)。



注意：本程式碼研究室需要使用實體裝置進行測試

TensorFlow 是多用途的機器學習架構。TensorFlow 的用途十分廣泛，從在雲端叢集訓練大型模型，到在手機等嵌入式系統上在本機執行模型，都能派上用場。

本程式碼實驗室會使用 [TensorFlow Lite](#)，在 Android 裝置上執行圖片辨識模型。

安裝 Android Studio 4.1 以上版本

如果尚未安裝，請在訓練 TensorFlow Lite 模型時，[下載並安裝 Android Studio 4.1 以上版本](#)。

課程內容

- 如何使用 [TensorFlow Lite Model Maker](#) 訓練自己的自訂圖片分類器。
- 如何使用 Android Studio 匯入 TensorFlow Lite 模型，並使用 CameraX 將自訂模型整合至 Android 應用程式。
- 瞭解如何在手機上使用 GPU 加速模型。

建構目標

這個簡單的相機應用程式會執行 TensorFlow 圖像辨識程式，辨識花朵。



授權：[可自由使用](#)

步驟 2 · 約 5 分鐘

2. 使用 Colab 訓練花卉辨識器

開始模型訓練前，請先下載並安裝 [Android Studio 4.1](#) 以上版本。

開啟 [Colab](#)，瞭解如何使用 Keras 訓練分類器，透過 TensorFlow Lite 遷移學習辨識花卉。

請務必：

1. 下載訓練好的模型 (model.tflite)，以及
2. 將 FlowerModel.tflite 檔案重新命名

再繼續操作。

部分瀏覽器會封鎖下載內容：

n/flowers_tf_lite.ipynb#scrollTo=GE4w-9S410Dk

Download blocked ☆

你可能需要授予 Colab 下載檔案的權限：

...lab.googleusercontent.com wants to



Download multiple files

Block

Allow

步驟 3 · 約 1 分鐘

3. 設定工作目錄

複製 Git 存放區

下列指令會複製包含本程式碼研究室所需檔案的 Git 存放區：

```
git clone https://github.com/hoitab/TFLClassify.git
```

接著，前往您剛複製存放區的目錄。您將在本程式碼研究室的其餘部分使用這個專案：

```
cd TFLClassify
```

步驟 4 · 約 5 分鐘

4. 設定 Android 骨架應用程式



安裝 Android Studio 4.1 以上版本

如果尚未安裝，請[安裝 Android Studio 4.1 以上版本](#)。

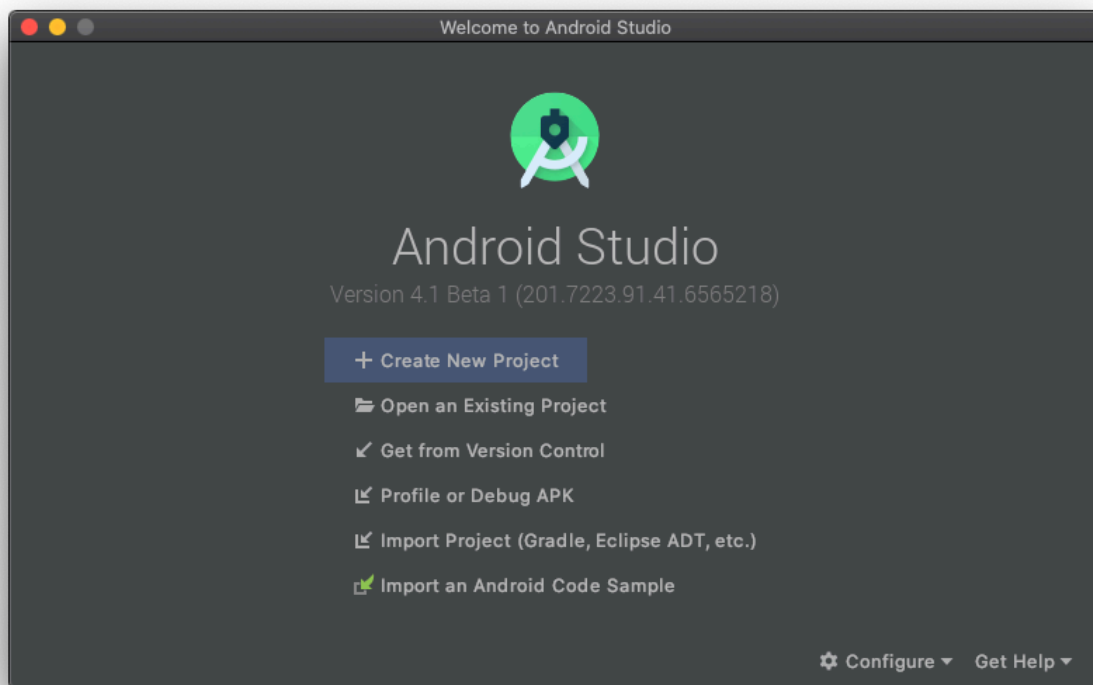
在 Android Studio 中開啟專案

按照下列步驟，使用 Android Studio 開啟專案：

1. 開啟 Android Studio



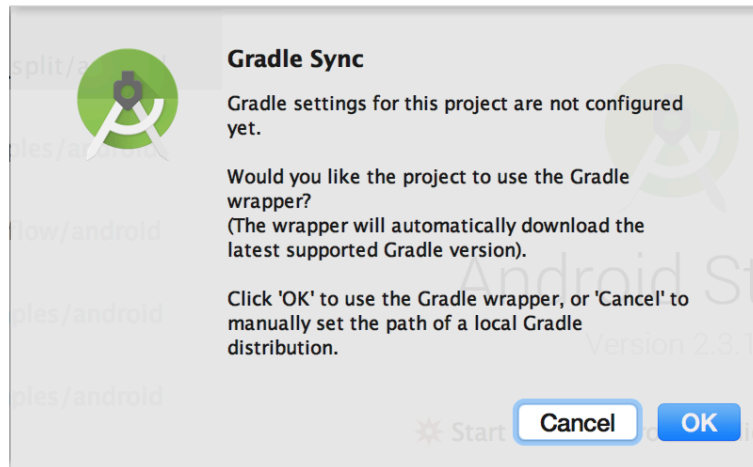
。載入完成後，請從這個彈出式視窗選取「Open an Existing project」：



2. 在檔案選取器中，從工作目錄選擇 `TFLClassify/build.gradle`。

專案包含兩個模組：`start` 和 `finish`。如果遇到問題，請參閱 `finish` 模組取得提示。

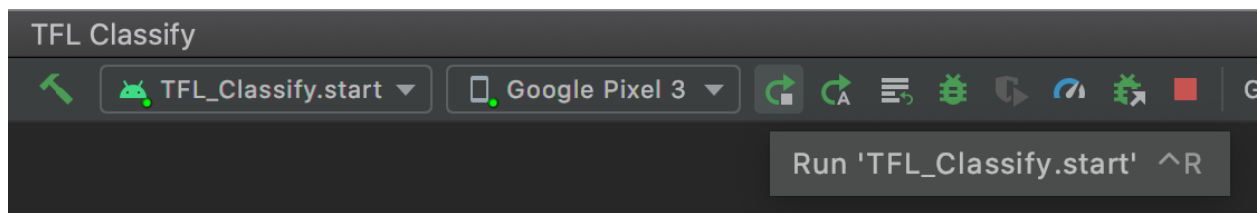
3. 首次開啟專案時，系統會顯示「Gradle Sync」彈出式視窗，詢問是否要使用 Gradle Wrapper。按一下 [確定]。



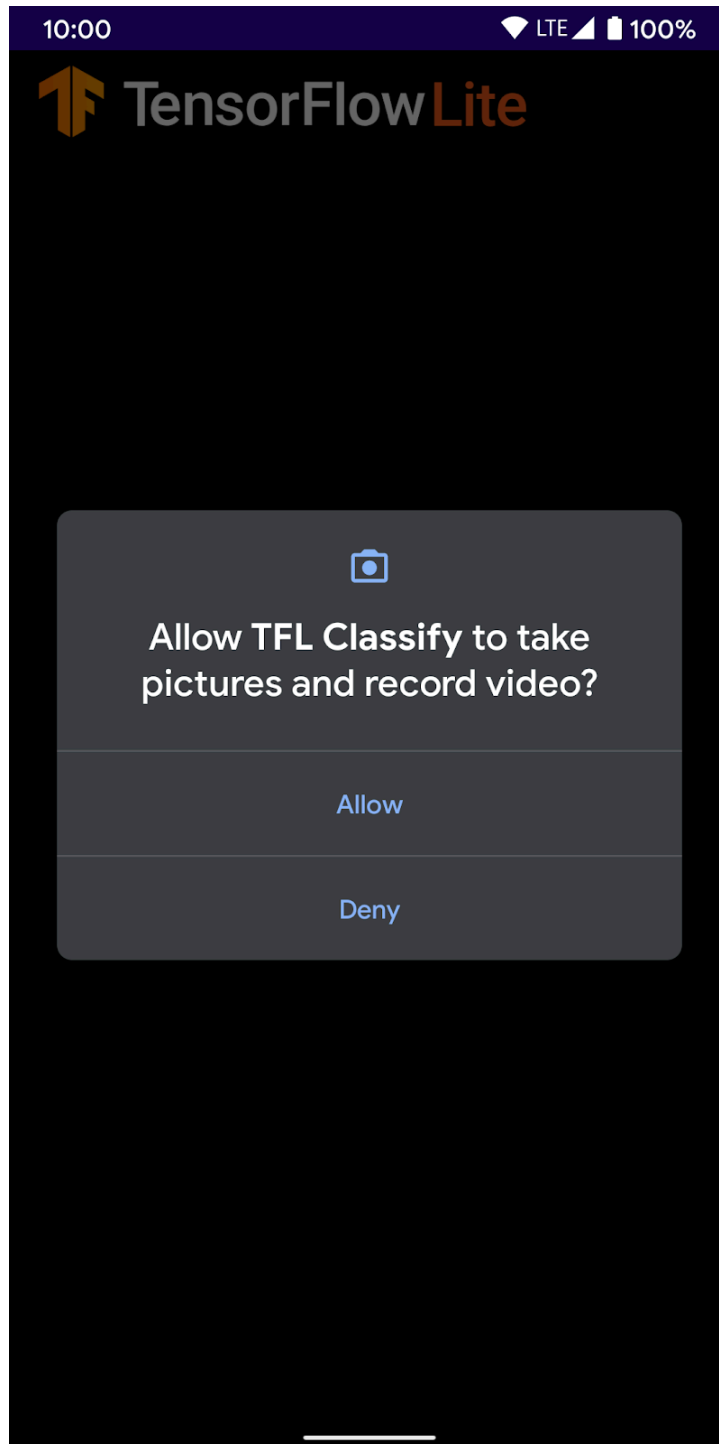
4. 如果尚未啟用手機的開發人員模式和 USB 偵錯功能，請啟用。這只需要設定一次。請按照[這些說明](#)操作。
5. 專案和手機都準備就緒後，選取 `TFL_Classify.start` 並按下工具列上的執行按鈕



，即可在實際裝置上執行專案：



6. 現在請允許 Tensorflow Demo 存取攝影機：



7. 手機上會顯示下列畫面，其中隨機數字會取代實際結果。

10:00

LTE 100%

 TensorFlow Lite

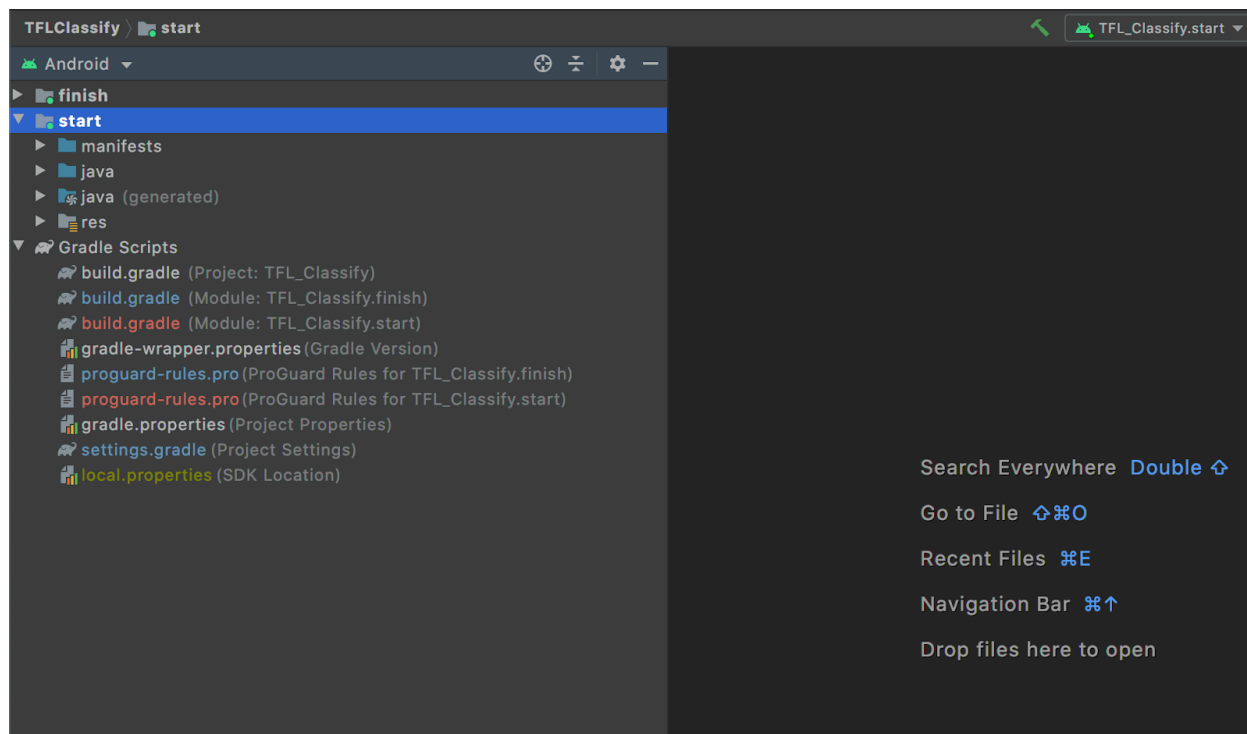


Fake label 0	91.1%
Fake label 1	54.5%
Fake label 2	45.3%

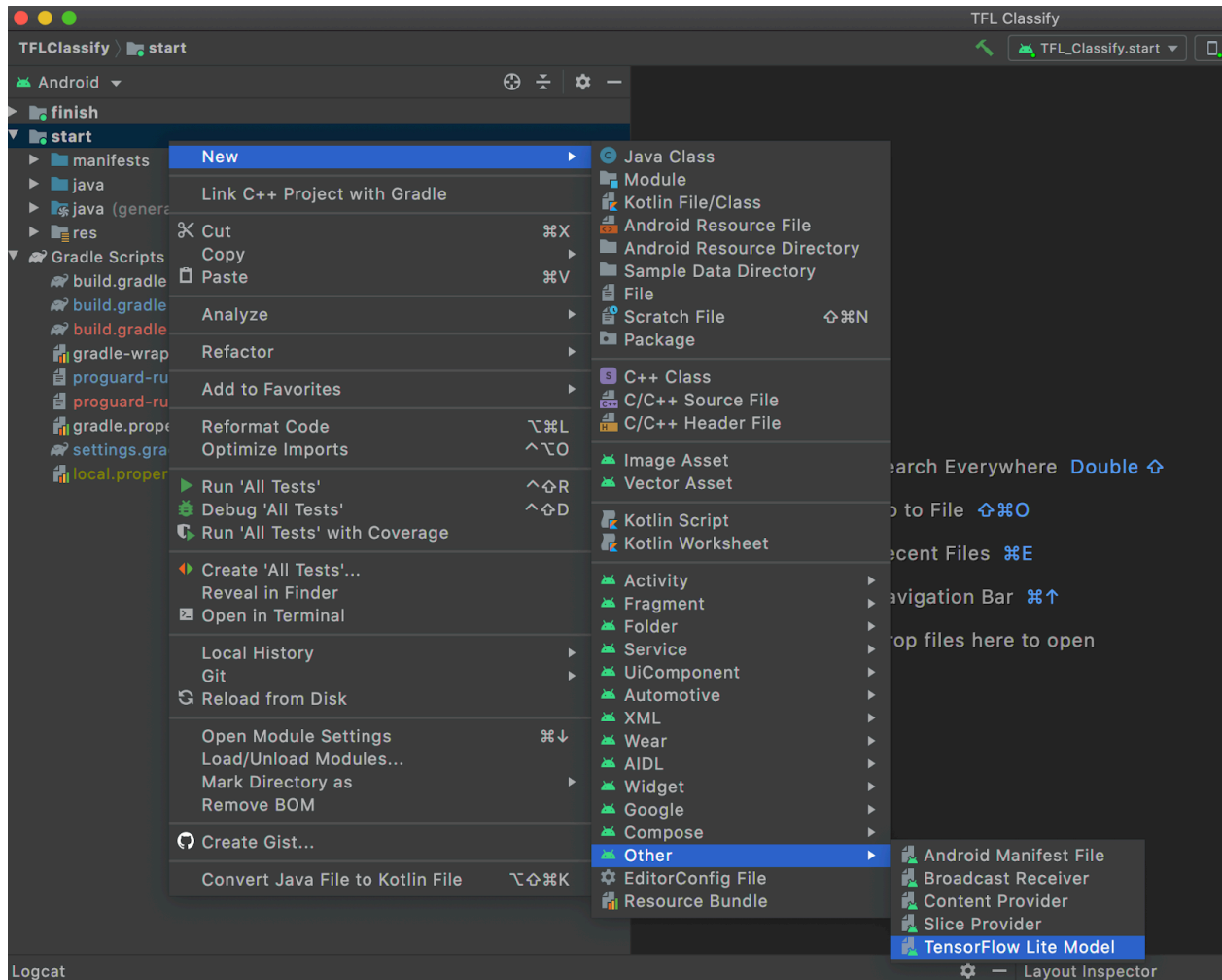
步驟 5 · 約 5 分鐘

5. 將 TensorFlow Lite 新增至 Android 應用程式

1. 在左側的專案探索工具中選取 `start` 模組：

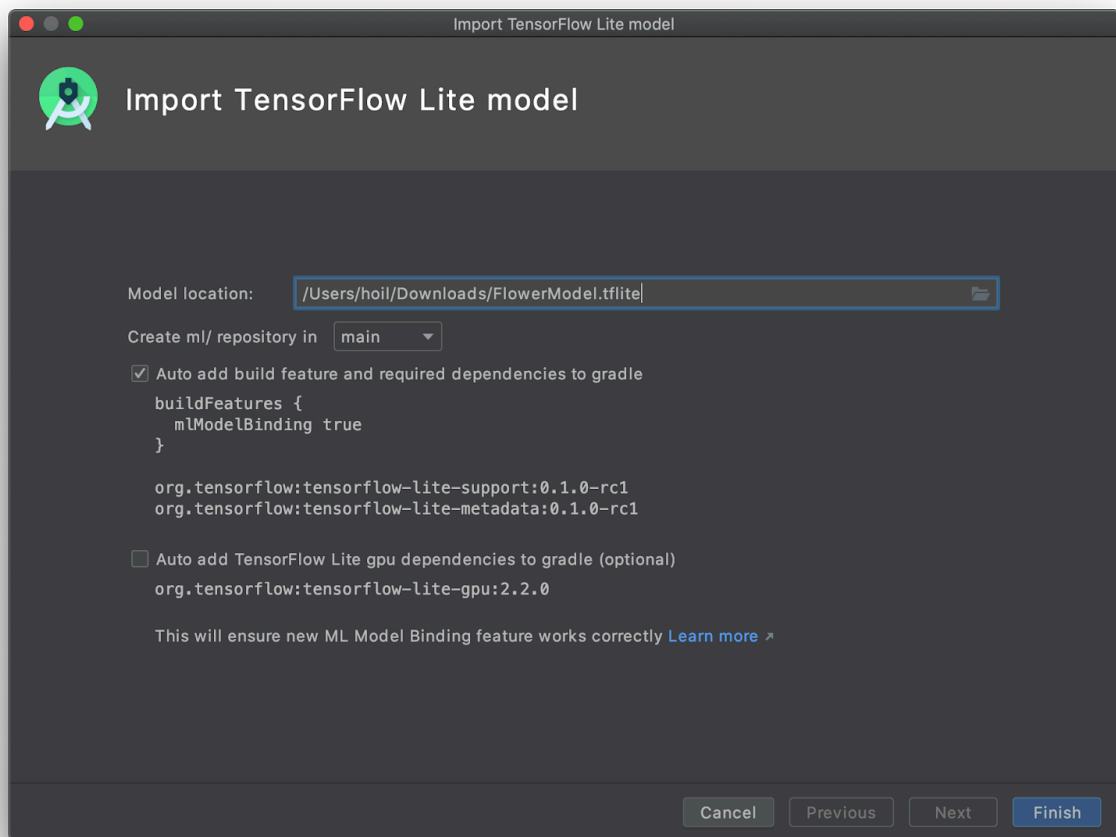


2. 在 `start` 模組上按一下滑鼠右鍵，或依序點選 `File`、`New`、`Other` 和 `TensorFlow Lite Model`



3. 選取先前下載自訂訓練 `FlowerModel.tflite` 的模型位置。

請注意，工具會代表您設定模組對機器學習模型繫結的依附元件，並自動將所有依附元件插入 Android 模組的 `build.gradle` 檔案。



4. 按一下「Finish」。
5. 最後會看到以下內容。FlowerModel.tflite 已順利匯入，並顯示模型的高階資訊，包括輸入 / 輸出內容，以及一些入門程式碼範例。

FlowerModel.tflite

Model

Name

efficientnet_lite0

Description

Identify the most prominent object in the image from a set of 5 categories.

Version

v1

Author

TensorFlow

License

Apache License. Version 2.0 <http://www.apache.org/licenses/LICENSE-2.0>.

Tensors

Inputs

Name	Type	Description	Shape	Mean / S...	Min / ...
image	Image <float32>	Input image to be classified. The expected image is 224 x 224, with three channels (red, blue, and green) per pixel. Each value in the tensor is a single byte between 0 and 1.	[1, 224, 224, 3]	[0] / [255]	[0] / [1]

Outputs

Name	Type	Description	Shape	Mean / S...	Min / ...
probability	Feature <float32>	Probabilities of the 5 labels respectively.	[1, 5]	[0] / [1]	[0] / [1]

Sample Code

Kotlin

Java

```
val model = FlowerModel.newInstance(context)

// Creates inputs for reference.
val image = TensorImage.fromBitmap(bitmap)

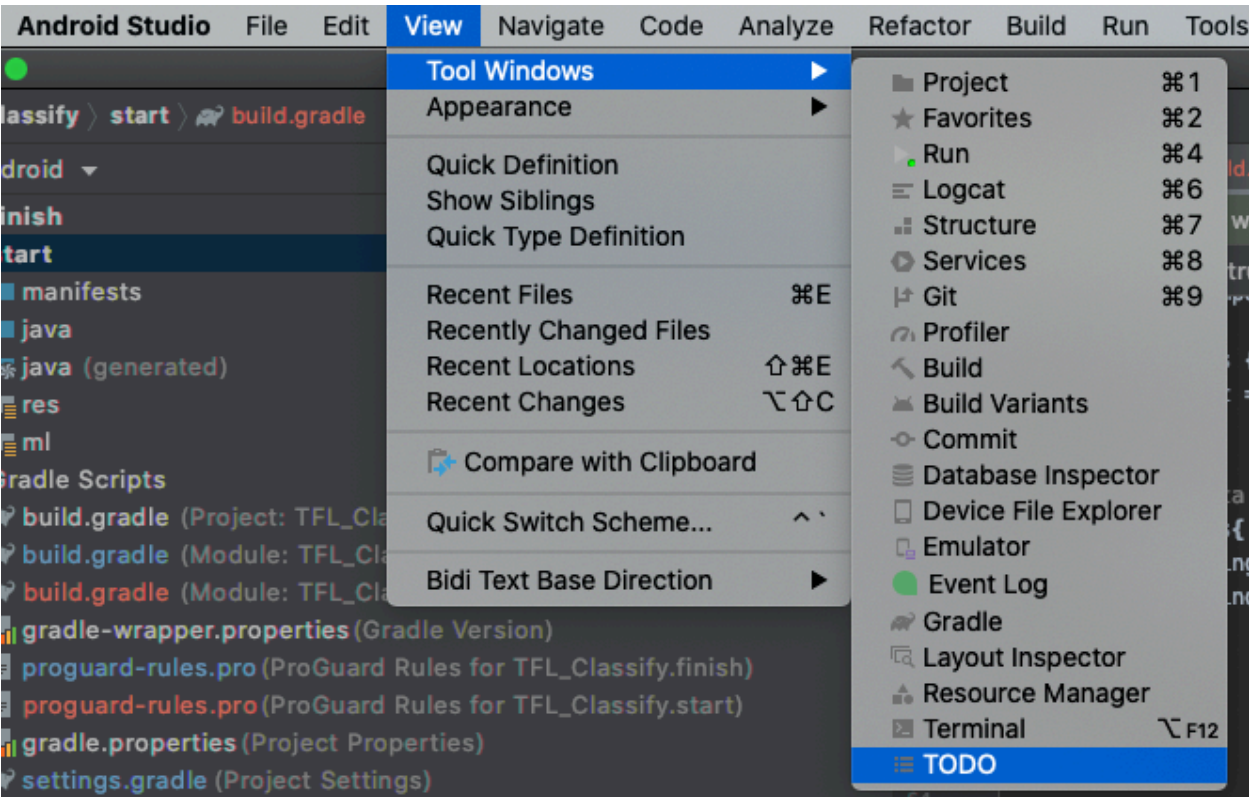
// Runs model inference and gets result.
val outputs = model.process(image)
val probability = outputs.probabilityAsCategoryList

// Releases model resources if no longer used.
model.close()
```

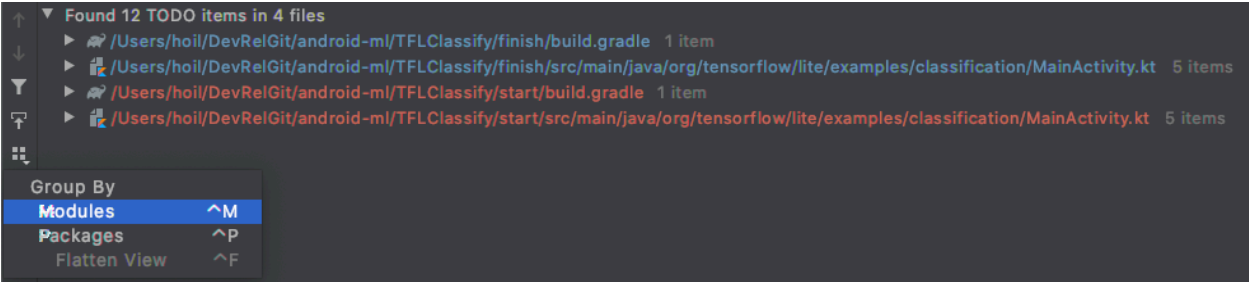
6. 選用：查看所有待辦事項清單

透過待辦事項清單，您可以輕鬆前往需要更新程式碼研究室的确切位置。您也可以 Android 專案中使用，提醒自己日後要完成的工作。您可以使用程式碼註解新增待辦事項，並輸入關鍵字 `TODO`。如要存取待辦事項清單，請採取下列任一做法：

1. 如要瞭解我們接下來要做什麼，最好的方法就是查看 `TODO` 清單。如要這麼做，請依序選取頂端選單列中的 `View > Tool Windows > TODO`。



2. 根據預設，這個視窗會列出所有模組中的所有 `TODO`，因此可能會造成一些混淆。按一下「TODO」面板側邊的「分組依據」按鈕，然後選擇 `Modules`，即可只排序開始的待辦事項。



3. 展開開始模組下的所有項目：

```
▼ Found 12 TODO items in 4 files
▶ TFL_Classify.finish 6 items
▼ TFL_Classify.start 6 items
  ▼ /Users/hoil/DevRelGit/android-ml/TFLClassify/start/build.gradle 1 item
    (85, 8) // TODO 5: Optional GPU Delegates
  ▼ /Users/hoil/DevRelGit/android-ml/TFLClassify/start/src/main/java/org/tensorflow/lite/examples/classification/MainActivity.kt 5 items
    (209, 12) // TODO 6. Optional GPU acceleration
    (211, 12) // TODO 1: Add class variable TensorFlow Lite Model
    (217, 16) // TODO 2: Convert Image to Bitmap then to TensorImage
    (219, 16) // TODO 3: Process the image using the trained model, sort and pick out the top results
    (221, 16) // TODO 4: Converting the top probability items into a list of recognitions
```

7. 使用 TensorFlow Lite 執行自訂模型

1. 按一下 TODO 清單中的 TODO 1，或開啟 **MainActivity.kt** 檔案並找出 TODO 1，然後新增下列程式碼行來初始化模型：

```
private class ImageAnalyzer(ctx: Context, private val listener: RecognitionListener) :
    ImageAnalysis.Analyzer {

    ...
    // TODO 1: Add class variable TensorFlow Lite Model
    private val flowerModel = FlowerModel.newInstance(ctx)

    ...
}
```

2. 在 CameraX Analyzer 的 analyze 方法內，我們需要將影像辨識輸入 `ImageProxy` 轉換為 `Bitmap`，並建立一個推論程序所需的 `TensorImage` 物件。

請注意，目前工具所用圖像須以 `Bitmap` 格式輸入。

- 這表示若輸入內容是檔案而非 `ImageProxy`，您可以將其做為 `Bitmap` 物件直接提供給 `fromBitmap` 方法。
- 如要瞭解 `ImageProxy` 如何轉換為 `Bitmap`，請參閱 `toBitmap` 方法和 `YuvToRgbConverter`。我們預期這些方法的需求是暫時的，因為團隊正努力支援 `ImageProxy`。 `TensorImage`

```
override fun analyze(imageProxy: ImageProxy) {
    ...
    // TODO 2: Convert Image to Bitmap then to TensorImage
    val tfImage = TensorImage.fromBitmap(toBitmap(imageProxy))
    ...
}
```

3. 處理圖片，然後對結果執行下列作業：

- 依屬性 `score` 下的機率降序排序結果，機率最高的結果會排在最前面。
- 根據常數 `MAX_RESULT_DISPLAY` 取得前 k 個結果。您可以視需要變更這個變數的值，以取得更多或更少的結果。

```

override fun analyze(imageProxy: ImageProxy) {
    ...
    // TODO 3: Process the image using the trained model, sort and pick out the top results
    val outputs = flowerModel.process(tfImage)
        .probabilityAsCategoryList.apply {
            sortByDescending { it.score } // Sort with highest confidence first
        }.take(MAX_RESULT_DISPLAY) // take the top results

    ...
}

```

4. 將排序及篩選結果轉換為資料物件 `Recognition`，準備透過[資料繫結](#)供 `RecyclerView` 使用：

```

override fun analyze(imageProxy: ImageProxy) {
    ...
    // TODO 4: Converting the top probability items into a list of recognitions
    for (output in outputs) {
        items.add(Recognition(output.label, output.score))
    }
    ...
}

```

5. 註解或刪除下列幾行，這些程式碼有助於產生我們之前看到的虛假結果：

```

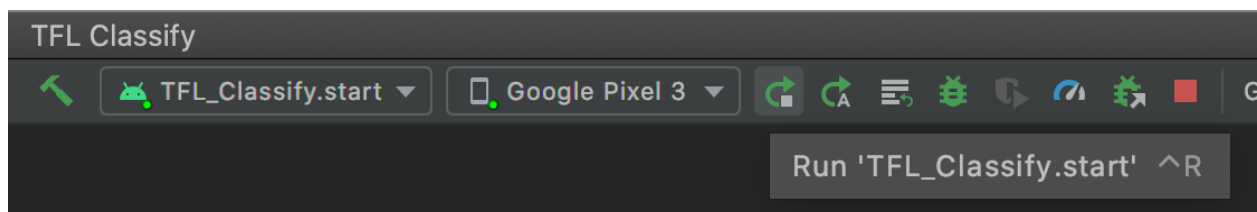
// START - Placeholder code at the start of the codelab. Comment this block of code out.
for (i in 0..MAX_RESULT_DISPLAY-1){
    items.add(Recognition("Fake label $i", Random.nextFloat()))
}
// END - Placeholder code at the start of the codelab. Comment this block of code out.

```

8. 選取 `TFL_Classify.start`，然後按下工具列上的「執行」按鈕



，在實體裝置上執行應用程式：



9. 手機上會顯示以下畫面，其中隨機數字會取代實際結果：



步驟 8 · 約 5 分鐘

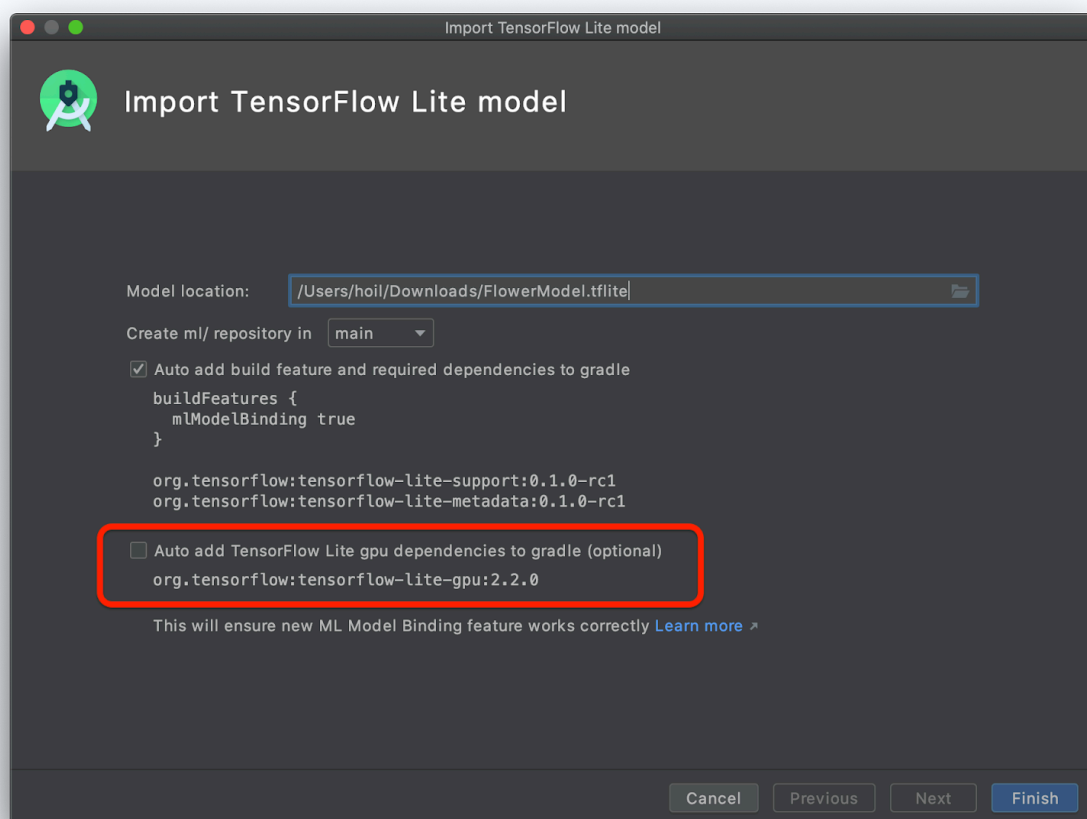
8. 選用：使用 GPU 委派加速推論

TensorFlow Lite 支援多種硬體加速器，可加快行動裝置的推論速度。[GPU](#) 是 TensorFlow Lite 可透過委派機制使用的其中一種加速器，而且相當容易使用。

1. 開啟 `start` 模組下的 `build.gradle`，或點選 TODO 清單下的 TODO 5，然後新增下列依附元件：

```
// TODO 5: Optional GPU Delegates  
implementation 'org.tensorflow:tensorflow-lite-gpu:2.3.0'
```

注意：在本程式碼研究室中，我們是手動新增這項匯入作業，但您在專案中執行這項操作時，只要勾選匯入畫面中的第二個方塊，即可新增 GPU 加速依附元件：



2. 返回 `MainActivity.kt` 檔案，或按一下 TODO 清單中的 TODO 6。將 `flowerModel` 的簡單啟動作業替換為下列項目：取得 GPU 相容性清單的執行個體，並根據是否為列出的相容 GPU 初始化 GPU。否則，請啟動 4 個 CPU 執行緒來執行模型：

```
private class ImageAnalyzer(ctx: Context, private val listener: RecognitionListener) :
    ImageAnalysis.Analyzer {
    ...

    // TODO 1: Add class variable TensorFlow Lite Model
    // Initializing the flowerModel by lazy so that it runs in the same thread when the process
    // method is called.
    private val flowerModel: FlowerModel by lazy{

        // TODO 6. Optional GPU acceleration
        val compatList = CompatibilityList()

        val options = if(compatList.isDelegateSupportedOnThisDevice){
            Log.d(TAG, "This device is GPU Compatible ")
            Model.Options.Builder().setDevice(Model.Device.GPU).build()
        } else {
            Log.d(TAG, "This device is GPU Incompatible ")
            Model.Options.Builder().setNumThreads(4).build()
        }

        ...
    }
}
```

注意：有多個 `Model` 物件，請選擇要匯入的物件 `org.tensorflow.lite.support.model.Model`

3. 在方法輸入內容中新增 `options`，即可變更模型初始值設定，以使用這項設定：

```
private class ImageAnalyzer(ctx: Context, private val listener: RecognitionListener) :
    ImageAnalysis.Analyzer {

    private val flowerModel: FlowerModel by lazy{

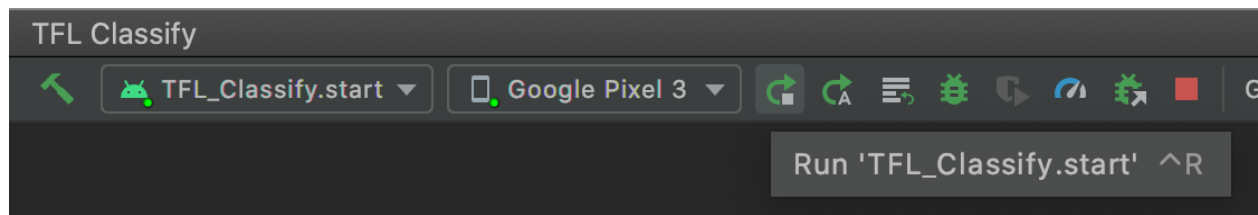
        ...

        // Initialize the Flower Model
        FlowerModel.newInstance(ctx, options)
    }
}
```

10. 選取 `TFL_Classify.start`，然後按下工具列上的「執行」按鈕



，在實體裝置上執行應用程式：



在中高階行動裝置上，GPU 的速度遠比 CPU 快。低階裝置的 GPU 速度通常較慢，因此加速效果會有所不同。

步驟 9 · 約 0 分鐘

9. 後續步驟

如需更多資訊，請參閱下列連結：

- 試用 tfhub.dev 上與機器學習模型繫結相容的其他 TFLite 模型。
 - 如要進一步瞭解 TFLite，請參閱 tensorflow.org 上的文件和[程式碼存放區](#)。
 - 您可以嘗試其他 [TFLite 預先訓練模型](#)，包括語音啟動字詞偵測器和裝置端智慧回覆。
 - 如要進一步瞭解 TensorFlow，請參閱[入門指南](#)文件。
-